URL Shortener

“Trí Tri - Tri Trí”



Outline

1. Understand The Problem
2. High Level Design
3. Low Level Design

1. Understand The Problem

1.1. Functional Requirements

- **URL shortening Service similar to TinyURL, or Bitly**
- **The same long URL can receive multiple different short URLs.**
- **Collision-free.**
- **Non-predictable.**
- Client can customize the short URL.
- The short URL should be web crawler friendly (SEO).
- The short URL should support analytics.

1.2. Capacity

Questions:

1. What is the average number of writes per day?
2. What is the read:write ratio?
3. What is the length of a short URL?
4. How many years should the short URL be persisted?

Answers:

1. 10M writes per day
2. 100:1
3. At most 8 characters
4. 4 years

1.3. Non-functional Requirements

- **Availability**
- **Low latency**
- Scalability
- Durability

2. Estimate

2. Estimate

- 10M writes per day → 10^7 TPD / 10^5 seconds = 10^(7-5) = 100 TPS
- Read:write = 100:1 → QPS = TPS * 100 = 10K QPS
- Storage for 4 years: ~ 17.6 TB
- Replication Factor: 3 → total storage = 3 * 17.6 TB = 52.8 TB
- Cache
  - TTL: 1 day
  - Size: 2.2 GB

3. High Level Design

Let's Design by Yourself

(mini interview)

Abstract Process

- Process: A long URL → short URL in binary (small) → encoded for readability
- Problems:
  - Encoding
  - Generating short URL in binary

Encoding

- Base36: a-z 0-9
- Base62: a-z A-Z 0-9
- Base64: base62 + /

- Choose Base62
- Reasons:
  - The higher number of chars in dictionary, the lower length of encoding output.
  - Friendly with web URL

Generate Short URL / Random ID

- Algorithms: UUID, CSPRNG
- Pros:
  - Simple
  - Scalability
  - Avoiding sequential guessing
- Cons:
  - The probability of **collisions** is high
  - Coordination between servers is required to prevent a collision

→ Don’t use Random ID here

Generate Short URL / Hash

-

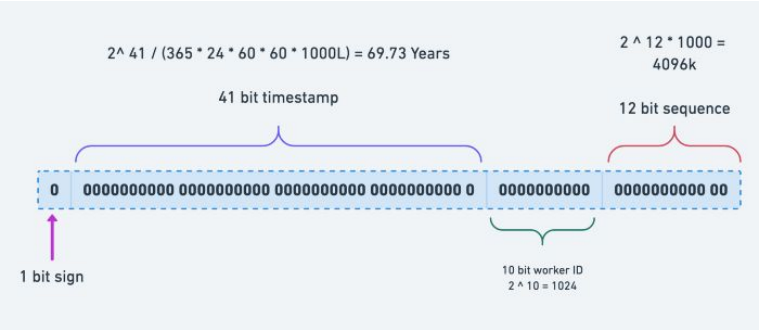
- Pros:
  - Scalability
- Cons:
  - Predictable output
  - Higher **probability** of a collision

→ Don’t use Hash here

Generate Short URL / Snowflake ID

-
- Pros:
  - Scalability
- Cons:
  - **Long output**: 64bits is encoded in Base62 → at least 11 characters
  - the probability of collisions is high due to the overlapping bits

→ Don’t use Snowflake ID here



$62^n \geq 2^{64}$

Taking the logarithm base 62:

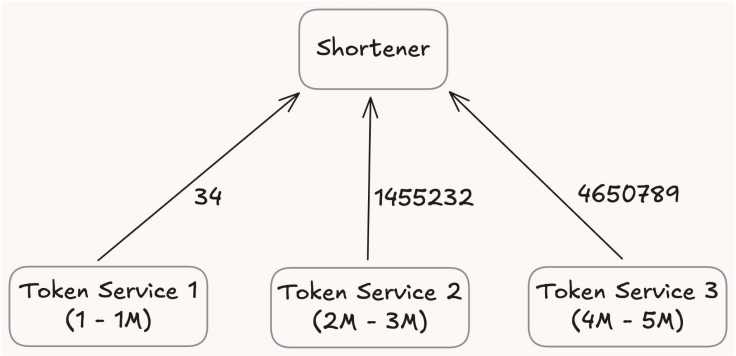
$n \geq \log_{62}(2^{64})$

Using the logarithmic change-of-base formula:

$$n \geq \frac{\log_{10}(2^{64})}{\log_{10}(62)}$$

Generate Short URL / Token Range

-
- Pros:
  - Scalability
- Cons:
  - Predictability
  - How to assign a range to a token service automatically and without overlapping
    - Zookeeper Coordinator



→ Use Token Range here

Storage

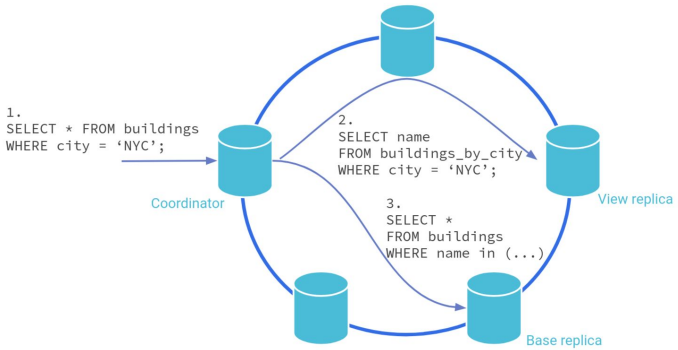
- NoSQL for table: URL
  - Non-relational data, No need for complex joins
  - Scalable
  - High performance
  - Recommend: ScyllaDB
- Relational DB for tables: User, Subscription
  - Relational data
  - ACID compliance
  - Strict schema
  - Recommend: Postgres

Partition Key

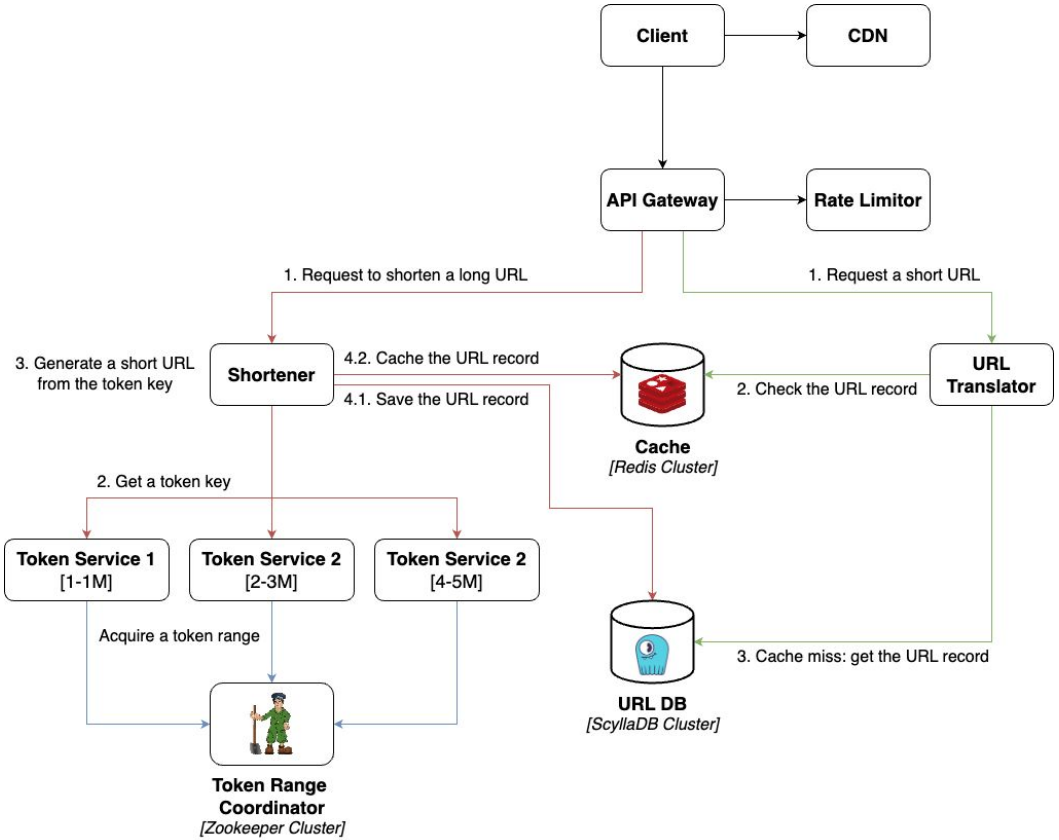
- Table URL
  - Primary key: shorten_url
  - Partition key: shorten_url
  - Strategy: hash

New Requirement: Long URL - Short URL: 1-1

- Need a operation to check the existing one
- Solution 1: Bloom Filter
  - False positives: BF tells yes → DB says no
  - Operation overhead
  - Not recommended
- Solution 2: Global Secondary Index
  - Easy: Transparent to app
  - Scalability
- Indexing on long URL → ineffficient
  - Indexing on hash(long_url)

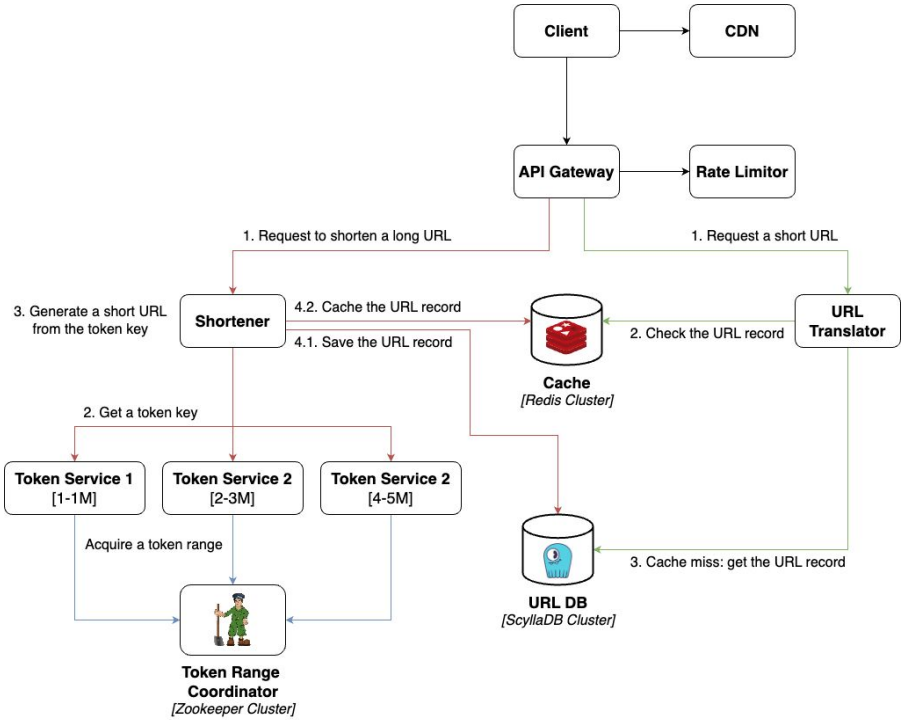


Final Design



Recap

- Process: Long URL
  → Short URL in binary (Token Range)
  → Encode (Base62)
- Generating: Avoid collision, the small size of output.
- Use Zookeeper in order that Token Ranges don’t overlap each other.



Homework

-

References

-

Thank you 🙏

